

AUDITORIA DE SEGURANÇA · SPA + SUPABASE

Relatório de Auditoria de Segurança — FrotaLog

Data: 21 de setembro de 2026

Projeto: FrotaLog (package.json name: frota-log)

Escopo: repositório completo em C:\Users\gabri\Documents\FrotaLog — código TypeScript/React, políticas SQL, seeds, CI e histórico git. Sem Docker, Helm ou Terraform no repositório.

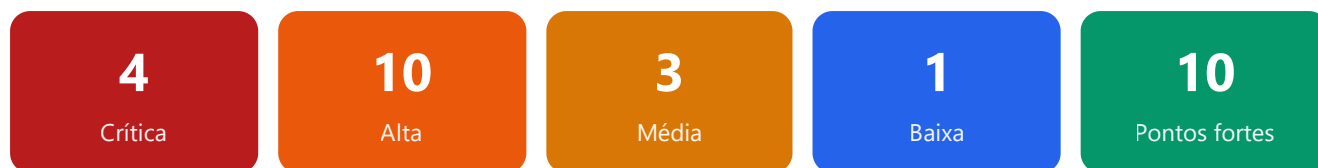
Nota metodológica — mapeamento das cinco categorias à stack

- **Linguagem / frontend:** TypeScript, React 19, Vite 8, Tailwind 4, React Router 7.
- **Backend:** não há API própria. O cliente chama o PostgREST/Auth do Supabase. A fronteira de segurança é o *Row Level Security* do Postgres, com defesa em profundidade em TenantScopeService e use-cases (código de navegador, contornável).
- **1. Banco sem tranca** → RLS ausente, USING (true), SELECT só com auth.uid() IS NOT NULL, ou listagens sem empresa_id.
- **2. Permissão no navegador** → gates canManage* / ProtectedRoute / classes Motorista cruzados com políticas RLS e com os use-cases.
- **3. IDOR** → todo handler de repositório que recebe id (path da SPA ou .eq('id')) e as políticas UPDATE/DELETE correspondentes. Não há routers HTTP para percorrer; o inventário é o conjunto de repositórios Supabase.
- **4. Chaves expostas** → hardcoded, fallbacks env || default, seeds, CI, histórico git e bundle frontend. Sem charts/compose.
- **5. XSS** → sinks React (dangerouslySetInnerHTML, eval, URLs em href/src) e HTML de e-mail/templates no backend (inexistente).

Achados apenas com evidência no código. A instância Supabase hospedada não foi consultada: a explorabilidade de F-03/F-04/F-05 depende de qual script SQL foi aplicado no projeto.

1. Resumo executivo

Foram confirmados **18 achados** no código: 4 críticos, 10 altos, 3 médios e 1 baixo. O risco central é a combinação de **chave anon pública no frontend** com **RLS incompleto ou permissivo** e **escalação de perfil no signup**.

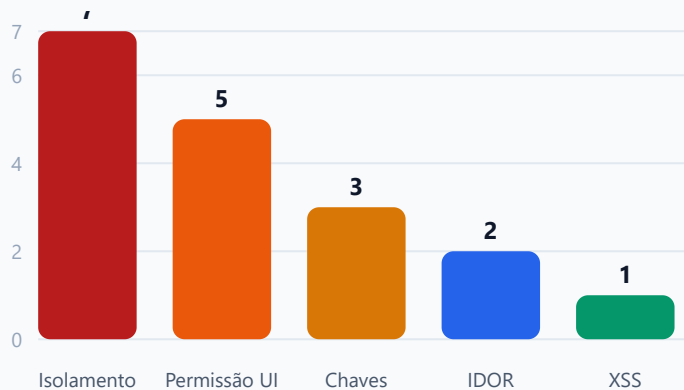


Por severidade



- Crítica (4)
- Alta (10)
- Média (3)
- Baixa (1)

Por categoria



2. Pontos fortes e pontos fracos

O que está protegido

TenantScopeService força empresa do perfil. `src/domain/services/TenantScopeService.ts:17–36` — não-`super_admin` ignora `empresald` pedido e usa `profile.empresa_id`.

rls_rbac_policies.sql combina tenant + papel nas 5 tabelas centrais.

`supabase/rls_rbac_policies.sql:126–266` — INSERT/UPDATE/DELETE de veículos, manutenções e alertas exigem `gestor/gerente` (e `mecânico` onde cabe) e `empresa_id`.

Relatório individual autorizado no caso de uso. `GetVeiculoReportUseCase.ts:55–64` + `VehicleReportAuthorizationService.ts:14–28` — motorista só no escopo de reserva; mecânico só com solicitação aprovada.

Aprovação de reserva e de relatório checada no application. `ReservaUseCases.ts:104–118` e `SolicitacaoRelatorioUseCases.ts:89–97` — `podeAprovar*` + `findByld` com `empresald`.

Criação de empresa só para Super Admin no application. `EmpresaApplicationService.ts:64–66` e `InviteKeyUseCases.ts:65–67`.

Token de convite com entropia criptográfica. `InviteKeyService.ts:10–13` — `crypto.randomUUID()`, formato `Chave_{Perfil}_{16 hex}`.

Seed de usuários exige service_role no ambiente. `scripts/seed-test-users.mjs:3–11` — aborta se `SUPABASE_SERVICE_ROLE_KEY` faltar; `service_role` não está no frontend.

deleteAll recusa exclusão global sem empresa_id. `SupabaseVeiculoRepository.ts:67–69`.

React escapa texto; sem HTML cru nem eval. Busca em `src/**` sem `dangerouslySetInnerHTML`, `innerHTML`, `eval` ou `new Function`. `hrefs` de notificação são constantes (`GetNotificationsUseCase.ts`). CSV escapa `aspas` (`VehicleReportExportService.ts:29–35`). `.env` não está no git.

ProtectedRoute e PermissionService existem para UX. `ProtectedRoute.tsx:45–70` e `App.tsx:42–48` — rotas `/empresas`, `/relatorios`, `/manutencoes`, `/historico` e `/alertas`. Isso não substitui RLS.

Riscos centrais

A fronteira é o RLS, e ela fura em vários scripts. O app é uma SPA com a anon key no bundle. Se `fix_database.sql` ou as políticas `dev_*` estiverem no banco, qualquer autenticado (e em parte o anon) lê e escreve todos os tenants.

O papel mora no navegador. Motorista é “somente leitura” nas classes de domínio e na UI, mas o schema canônico autoriza escrita por `empresa_id` apenas, e o trigger de signup aceita `super_admin` nos metadados.

Onboarding público vaza segredos de adesão. SELECT de todas as chaves ativas + catálogo de empresas para anon torna trivial entrar em qualquer empresa ou alimentar a escalação de perfil.

Senha e projeto reais no git. `admin@frotamaq.com / 123456` e o JWT anon do projeto `aechddecxuvysrngpbord` estão versionados.

3. Achados detalhados

Uma linha por achado. A coluna do meio traz caminho e linhas exatas (vários arquivos quando o mesmo defeito se repete).

Severidade	Arquivo:linha	Descrição
Crítica	supabase/frota_maq_recreate_complete.sql:212-237 migration_signup_empresa.sql:4-24 schema.sql:213-219 fix_database.sql:153-171 AuthContext.tsx:167-185	F-01 · Permissão no navegador O trigger SECURITY DEFINER <code>handle_new_user</code> grava perfil e <code>empresa_id</code> a partir de <code>raw_user_meta_data</code> , sem validar convite. <code>signup()</code> envia o perfil escolhido pelo cliente. Qualquer pessoa chama <code>supabase.auth.signup</code> com <code>perfil=super_admin</code> .
Crítica	supabase/frota_maq_recreate_complete.sql:303-314	F-02 · IDOR Política <code>usuarios_update</code> permite UPDATE do próprio registro (<code>id = auth.uid()</code>) sem travar a coluna <code>perfil</code> nem <code>empresa_id</code> . O usuário autentica e faz <code>update({perfil:'super_admin'})</code> ou troca <code>empresa_id</code> — escalação e salto de tenant, depois lê/altera objetos de outro inquilino por ID.
Crítica	fix_database.sql:230-304 src/lib/supabase.ts:139 e 101	F-03 · Isolamento de tenant GRANT ALL em todas as tabelas para <code>anon+authenticated</code> e políticas USING (true) / WITH CHECK (true), inclusive SELECT anônimo. O próprio cliente pede para executar este script se a tabela <code>usuarios</code> falhar.
Crítica	supabase/schema_reset_complete.sql:244-271	F-04 · Isolamento de tenant Políticas <code>dev_*</code> com USING (true) em empresas, usuarios, veiculos, manutencoes e alertas. Qualquer usuário autenticado lê e escreve todos os inquilinos.

Severidade	Arquivo:linha	Descrição
Alta	supabase/schema.sql:269-329	F-05 · Isolamento de tenant SELECT de veiculos, manutencoes e alertas usa apenas auth.uid() IS NOT NULL — sem empresa_id. IDOR cross-tenant: GET por UUID devolve o registro de qualquer empresa.
Alta	supabase/frota_maq_recreate_complete.sql:432-434 migration_rbac_invite_keys.sql:49-51	F-06 · Isolamento de tenant chaves_convite_public_validate permite SELECT a anon/authenticated com USING (ativa = TRUE), sem filtrar o token. Um cliente anônimo lista todas as chaves ativas, perfil e empresa_id.
Alta	supabase/migration_signup_empresa.sql:29-32	F-07 · Isolamento de tenant empresas_select_signup dá SELECT de todas as empresas ao papel anon (USING true). Catálogo completo de tenants (id, nome, slug, CNPJ) sem autenticação.
Alta	supabase/frota_maq_recreate_complete.sql:333-492	F-08 · Permissão no navegador No schema canônico, INSERT/UPDATE/DELETE de veículos, manutenções, reservas, convites e solicitações só exigem empresa_id do chamador. Motorista (UI somente leitura) cria, altera e apaga qualquer objeto do tenant via API Supabase.
Alta	supabase/migration_reservas_veiculos.sql:51-60	F-09 · IDOR reservas_update autoriza motorista_id = auth.uid() sem restringir colunas. O motorista atualiza status para APROVADO na própria reserva (autoaprovação por ID).
Alta	src/lib/supabase.ts:3-8	F-10 · Chaves expostas URL do projeto e JWT anon reais hardcoded, usados como fallback se VITE_* estiver vazio. Sem validação de startup que rejeite o default. A chave está no git desde o commit inicial. Explorável de imediato se o RLS estiver fraco (F-03 a F-08).
Alta	supabase/frota_maq_recreate_complete.sql:8-9 e 553 src/lib/testUsers.ts:3 scripts/seed-test-users.mjs:18-20	F-11 · Chaves expostas Super admin admin@frotamaq.com é criado com senha 123456. A senha aparece no SQL, no frontend e no seed. Se o script rodou no projeto hospedado, o login privilegiado é público.
Alta	src/application/use-cases/VeiculoUseCases.ts:38-40 ManutencaoUseCases.ts:15-17 src/services/vehicles.ts:47-56 src/services/maintenance.ts:39-48 VehiclesPage.tsx:124	F-12 · Permissão no navegador A UI esconde cadastro com canManageVehicles / canCreateMaintenance. CreateVeiculoUseCase e CreateManutencaoUseCase não verificam papel. O gate existe só no navegador; o servidor (RLS canônico) também não exige gestor.

Severidade	Arquivo:linha	Descrição
Alta	supabase/frota_maq_recreate_complete.sql:359-364 rls_rbac_policies.sql:176-181 Motorista.ts:10-11	F-13 · Isolamento de tenant SELECT de manutencoes (incluindo valor_total) é liberado a qualquer papel do tenant. Motorista tem podeVisualizarRelatorios()=false na UI, mas lê custos de toda a frota via from('manutencoes').
Alta	supabase/frota_maq_recreate_complete.sql:222-231 migration_signup_empresa.sql:17-18 AuthContext.tsx:169-184	F-14 · Isolamento de tenant empresa_id do cadastro vem dos metadados do Auth. Combinado com F-06/F-07 (IDs vazados), o atacante entra em qualquer tenant no signup, sem chave daquela empresa.
Média	supabase/rls_rbac_policies.sql:143-158	F-15 · Permissão no navegador veiculos_update_policy autoriza mecanico a UPDATE de qualquer coluna do veículo da empresa, não só quilometragem. A UI limita edição; o UPDATE direto altera placa, status, empresa_id (desde que o WITH CHECK passe).
Média	src/lib/testUsers.ts:3 src/lib/seedTestUsers.ts:22	F-16 · Chaves expostas TEST_USER_PASSWORD = '123456' entra no bundle Vite. Confirma a senha padrão das contas de demonstração para quem inspeciona o JS.
Média	src/services/vehicles.ts:75-125 VehiclesPage.tsx:124-137	F-17 · Permissão no navegador deleteAllVeiculos e seedTestVeiculos só aplicam TenantScopeService. A UI mostra os botões com canSeedTestData / canManageVehicles. Motorista autenticado chama as funções (ou o cliente Supabase) e apaga/popula a frota do tenant se o RLS canônico estiver ativo.
Baixa	src/pages/VehiclesPage.tsx:231-232 VehicleDetailPage.tsx:254-255	F-18 · XSS / inputs foto_url do banco vai direto para sem allowlist http(s). Não há dangerouslySetInnerHTML. Em navegadores modernos javascript: em img raramente executa; risco residual de URL maliciosa/tracking se a coluna existir (schemas antigos) e o UPDATE de veículo estiver aberto.

4. Recomendações priorizadas

P1 — Fechar escalação de perfil (F-01, F-02, F-14). Reescrever handle_new_user para não ler papel/tenant dos metadados; travar UPDATE de perfil e empresa_id; cadastro só via RPC de convite.

P1 — Único schema RLS endurecido (F-03, F-04, F-05, F-08, F-09, F-13). Auditar pg_policies no projeto hospedado; remover leftovers permissivos; unificar recreate + rls_rbac (tenant e papel); proibir autoaprovação de reserva.

P1 — Convites e empresas (F-06, F-07). RPC de validação por token; zero SELECT anon em `chaves_convite` e `empresas`.

P2 — Segredos (F-10, F-11, F-16). Remover fallbacks; falhar o boot; rotacionar anon key; trocar senha do admin; tirar 123456 do bundle.

P2 — Autorização nos use-cases (F-12, F-15, F-17). Checar papel antes do repositório; restringir colunas do UPDATE do mecânico.

P3 — foto_url (F-18). Allowlist https: no renderizador de imagem.

5. Cobertura da auditoria (o que foi verificado e está correto)

- Repositórios: `SupabaseVeiculoRepository`, `SupabaseManutencaoRepository`, `SupabaseReservaRepository`, `SupabaseSolicitacaoRelatorioRepository`, `SupabaseChaveConviteRepository`, `SupabaseEmpresaRepository`, `SupabaseAlertaRepository` (stub — tabela alertas ausente no schema canônico).
- Use-cases de reserva, relatório, convite e empresa *checam* permissão no application (ponto forte). Veículo create/update/delete e manutenção create *não* checam (F-12).
- Não há Docker, Helm, Terraform nem docker-compose. CI (`.github/workflows/ci.yml`) não injeta segredos.
- `.env` real não está no git; `.env.example` só tem placeholders. `SUPABASE_SERVICE_ROLE_KEY` não aparece no frontend.
- XSS: nenhum `dangerouslySetInnerHTML`, `innerHTML`, `eval`, `new Function` ou HTML de e-mail. Sem `DOMPurify` — e não há renderização de markdown/HTML além de `foto_url`.
- Categoria “API própria sem filtro de tenant” não se aplica como Express/Fastify; o equivalente auditado foi RLS + queries `supabase.from()`.

6. ISSUES PARA O GITHUB

Cada bloco abaixo está delimitado por `--- ISSUE n ---` e `--- FIM ISSUE n ---`, pronto para colar.

Issue 1 — [Segurança] Escalação de privilégio no cadastro e na atualização de perfil

Labels: **security, crítica** · Achados F-01, F-02, F-14

Copie o bloco abaixo e cole numa issue do GitHub.

```

--- ISSUE 1 ---
## Título
[Segurança] Escalação de privilégio no cadastro e na atualização de perfil

## Labels sugeridas
`security, crítica`

## Achados
F-01, F-02, F-14

## Descrição do problema e por que é explorável
O backend (trigger SECURITY DEFINER `handle_new_user` e a política `usuarios_update`) confia no cliente para definir `perfil` e `empresa_id`.

O frontend de cadastro até valida a chave de convite, mas isso roda só no navegador. Qualquer um abre o DevTools e chama:

```js
await supabase.auth.signUp({
 email: 'atacante@mail.com',
 password: 'qualquer',
 options: { data: { nome: 'X', perfil: 'super_admin', empresa_id: '<uuid vazado>' } }
})
```

O trigger grava `super_admin` ignorando RLS. Se o schema canônico estiver aplicado, um usuário já logado também faz `supabase.from('usuarios').update({ perfil: 'super_admin' }).eq('id', user.id)`.

Com `empresa_id` escolhido (IDs vazam em F-06/F-07), o atacante entra em outro tenant.

## Evidência
`supabase/frota_maq_recreate_complete.sql:212-237`
```sql
meta_perfil := NULLIF(TRIM(NEW.raw_user_meta_data->>'perfil'), '');
...
COALESCE(meta_perfil::public.perfil_usuario, 'motorista'::public.perfil_usuario),
meta_empresa
```

`supabase/frota_maq_recreate_complete.sql:303-314` – UPDATE com `id = auth.uid()` sem restrição de coluna.

`src/contexts/AuthContext.tsx:167-185` – `signUp` envia `perfil` e `empresa_id` do cliente.

## Impacto
Conta nova vira Super Admin (visão e escrita em todos os tenants) ou salta para empresa alheia. Quebra total do modelo multi-tenant e do RBAC.

## Sugestão de correção
- No trigger: ignorar `perfil`/`empresa_id` dos metadados. Sempre criar como `motorista` (ou perfil da chave validada no servidor).
- Validar convite numa RPC `SECURITY DEFINER` que recebe só o token, resolve empresa+perfil e insere o usuário.
- Em `usuarios`: política de UPDATE que permite só `nome`/campos inofensivos; `perfil` e `empresa_id` só por `super_admin` (ou trigger que rejeita mudança dessas colunas se não for service_role).
- Remover `upsertUsuarioProfile` que reescreve perfil a partir do cliente.

## Critérios de aceite
- [ ] signUp com metadata perfil=super_admin resulta em motorista (ou falha), nunca super_admin

```

```
- [ ] usuarios.update({perfil, empresa_id}) pelo próprio usuário é negado pelo RLS  
- [ ] Cadastro sem chave válida não associa empresa_id arbitrário  
- [ ] Teste automatizado cobre os três caminhos (signup, update próprio, metadata)  
  
--- FIM ISSUE 1 ---
```

Issue 2 — [Segurança] Scripts SQL com RLS permissivo (USING true / GRANT ALL a anon)

Labels: **security, crítica** · Achados F-03, F-04, F-05

Copie o bloco abaixo e cole numa issue do GitHub.

```

--- ISSUE 2 ---
## Título
[Segurança] Scripts SQL com RLS permissivo (USING true / GRANT ALL a anon)

## Labels sugeridas
`security, crítica`

## Achados
F-03, F-04, F-05

## Descrição do problema e por que é explorável
Há três scripts no repositório que, se executados no projeto Supabase, desligam o isolamento de tenant:

1. `fix_database.sql` – `GRANT ALL` a `anon` e `authenticated` + políticas `USING (true)` e SELECT anônimo em usuarios/veiculos/manutencoes/alertas. O app ainda instrui a rodá-lo (`src/lib/supabase.ts`).
2. `schema_reset_complete.sql` – políticas `dev_*` `USING (true)` em todas as tabelas.
3. `schema.sql` – SELECT de frota/manutenções/alertas com `auth.uid() IS NOT NULL`, sem `empresa_id`.

A fronteira real deste produto é o RLS (SPA fala direto com o PostgREST). Qualquer um desses scripts no banco hospedado permite listar e, nos dois primeiros, alterar dados de todos os inquilinos com a anon key pública.

## Evidência
`fix_database.sql:230-277`
```sql
GRANT ALL ON ALL TABLES IN SCHEMA public TO anon, authenticated;
CREATE POLICY "authenticated_all_usuarios"
 ON public.usuarios FOR ALL TO authenticated
 USING (true) WITH CHECK (true);
CREATE POLICY "anon_select_veiculos"
 ON public.veiculos FOR SELECT TO anon USING (true);
```

`schem_reset_complete.sql:244-247` – `dev_empresas_* USING (true)`.

`schem.sql:269-273` – `USING (auth.uid() IS NOT NULL)`.

## Impacto
Vazamento e adulteração cross-tenant (e, em fix_database, leitura anônima da frota e da tabela de usuários).

## Sugestão de correção
- Arquivar ou apagar `fix_database.sql` e as políticas `dev_*`. Trocar o aviso do cliente por um script único endurecido.
- Manter um único schema canônico com RLS por `empresa_id` + papel; CI/documentação apontam só para ele.
- Auditar o projeto Supabase atual (`pg_policies`) e dropar leftovers `dev_*`, `anon_select_*`, `authenticated_all_*`.
- SELECT sempre com `empresa_id = auth_user_empresa_id()` (exceto super_admin).

## Critérios de aceite
- [ ] Nenhum script versionado cria política USING (true) nem GRANT ALL a anon
- [ ] Consulta SQL no projeto hospedado: zero políticas dev_/anon_select_/authenticated_all_
- [ ] Usuário da empresa A não lê veículo/manutenção/alerta da empresa B por listagem nem por ID
- [ ] Papel anon não lê usuarios, veiculos, manutencoes, alertas

```

--- FIM ISSUE 2 ---

Issue 3 — [Segurança] Schema canônico sem RBAC nas escritas e IDOR por UUID

Labels: **security, alta** · Achados F-08, F-09, F-13

Copie o bloco abaixo e cole numa issue do GitHub.

```

--- ISSUE 3 ---
## Título
[Segurança] Schema canônico sem RBAC nas escritas e IDOR por UUID

## Labels sugeridas
`security, alta`

## Achados
F-08, F-09, F-13

## Descrição do problema e por que é explorável
`frota_maq_recreate_complete.sql` isola por tenant, mas as políticas de escrita de `veiculos`,
`manutencoes`, `reservas`, `chaves_convite` e `solicitacoes_relatorio` não exigem papel. Qualquer membro do
tenant (incluindo motorista) que conheça o UUID – e o SELECT do tenant entrega a lista – cria, altera ou
apaga o objeto.

Em `migration_reservas_veiculos.sql` o motorista ainda pode `UPDATE` a própria reserva e mudar `status`
para `APROVADO`.

O SELECT de `manutencoes` devolve `valor_total` a todos os papéis, embora a UI esconda relatórios
financeiros do motorista.

## Evidência
`frota_maq_recreate_complete.sql:333-338` (veículos; o mesmo padrão se repete até a linha 492)
```sql
CREATE POLICY "veiculos_insert" ON public.veiculos
 FOR INSERT TO authenticated
 WITH CHECK (
 public.auth_user_perfil() = 'super_admin'
 OR empresa_id = public.auth_user_empresa_id()
);
```

`migration_reservas_veiculos.sql:51-60` – `OR motorista_id = auth.uid()` no UPDATE sem restrição de coluna.

## Impacto
Motorista (ou mecânico) destrói a frota, inventa manutenções, autoaprova reserva e lê custos de toda a
empresa. A UI “somente leitura” não impede.

## Sugestão de correção
- Portar as restrições de papel de `rls_rbac_policies.sql` para o schema canônico (gestor/gerente escrevem
veículos; gestor/mecânico escrevem manutenções; só gestor aprova reserva/relatório).
- UPDATE de reserva pelo motorista: proibir mudança de `status` (coluna ou política).
- SELECT financeiro: view sem `valor_total` para motorista, ou política por papel.
- Repositórios: `findById`/`update`/`delete` sempre com `empresa_id` (já parcialmente feito).

## Critérios de aceite
- [ ] Motorista: INSERT/UPDATE/DELETE em veiculos e manutencoes retorna erro RLS
- [ ] Motorista: UPDATE reservas.set status=APROVADO retorna erro RLS
- [ ] Motorista: SELECT valor_total de manutenção de veículo sem reserva/aprovação falha ou vem mascarado
- [ ] Gestor da mesma empresa continua aprovando reserva e CRUD de frota

--- FIM ISSUE 3 ---

```

Issue 4 — [Segurança] Vazamento de chaves de convite e catálogo de empresas para anon

Labels: **security, alta** · Achados F-06, F-07

Copie o bloco abaixo e cole numa issue do GitHub.

```

--- ISSUE 4 ---
## Título
[Segurança] Vazamento de chaves de convite e catálogo de empresas para anon

## Labels sugeridas
`security, alta`

## Achados
F-06, F-07

## Descrição do problema e por que é explorável
Duas políticas públicas expõem o mecanismo de onboarding:

- `chaves_convite_public_validate` — `FOR SELECT TO anon, authenticated USING (ativa = TRUE)`. Não filtra pelo token informado. `from('chaves_convite').select('*')` devolve todos os tokens ativos, perfil e `empresa_id`.
- `empresas_select_signup` — `FOR SELECT TO anon USING (true)`. Lista id/nome/slug/CNPJ de todas as empresas.

Com a anon key do frontend (F-10), isso é explorável sem login e alimenta a escalção de F-01/F-14.

## Evidência
`frota_maq_recreate_complete.sql:432-434` e `migration_rbac_invite_keys.sql:49-51`
```sql
CREATE POLICY "chaves_convite_public_validate" ON public.chaves_convite
 FOR SELECT TO anon, authenticated
 USING (ativa = TRUE);
```

`migration_signup_empresa.sql:29-32`
```sql
CREATE POLICY "empresas_select_signup" ON public.empresas
 FOR SELECT TO anon
 USING (true);
```

## Impacto
Qualquer visitante obtém convites válidos (entra como motorista/mecânico de qualquer empresa) e o mapa de tenants.

## Sugestão de correção
- Trocar a política pública por RPC `validar_chave_convite(token text)` com `SECURITY DEFINER` que devolve só empresa_id+perfil se o token existir e estiver ativo; não expor a tabela.
- Remover SELECT anon em `empresas`. O cadastro não precisa listar empresas se a chave já resolve o tenant.
- GRANT SELECT em `chaves_convite` somente a `authenticated` com filtro de tenant, nunca a `anon` em todas as linhas ativas.

## Critérios de aceite
- [ ] Cliente anon: from('chaves_convite').select('*') retorna 0 linhas
- [ ] RPC/validação com token certo devolve empresa+perfil; token errado não vaza outros tokens
- [ ] Cliente anon: from('empresas').select('*') retorna 0 linhas

--- FIM ISSUE 4 ---

```

Issue 5 — [Segurança] Credenciais padrão e chave anon hardcoded no frontend

Labels: **security, alta** · Achados F-10, F-11, F-16

Copie o bloco abaixo e cole numa issue do GitHub.

```

--- ISSUE 5 ---
## Título
[Segurança] Credenciais padrão e chave anon hardcoded no frontend

## Labels sugeridas
`security, alta`

## Achados
F-10, F-11, F-16

## Descrição do problema e por que é explorável
`src/lib/supabase.ts` embute a URL `https://aechddecxuvysrgpbord.supabase.co` e o JWT anon do projeto. Se
`VITE_SUPABASE_*` estiver vazio, esse default é usado – padrão `VAR || hardcoded` sem rejeição no startup.

A chave anon é pública por desenho do Supabase somente se o RLS estiver correto. Aqui o RLS tem falhas
críticas; o default committed no git (desde `316ee59`) aponta para o projeto real.

O schema canônico cria `admin@frotamaq.com` / `123456`. A mesma senha está em `testUsers.ts` (bundle),
seeds e scripts.

## Evidência
`src/lib/supabase.ts:3-8`
```ts
export const SUPABASE_URL = 'https://aechddecxuvysrgpbord.supabase.co'
export const SUPABASE_ANON_KEY = 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...'
const supabaseUrl = import.meta.env.VITE_SUPABASE_URL?.trim() || SUPABASE_URL
```

`frota_maq_recreate_complete.sql:8-9, 553` – `crypt('123456', gen_salt('bf'))`.

Histórico: `git log` em `src/lib/supabase.ts` mostra o valor no commit inicial. Nenhum `.env` real está
versionado (ponto positivo).

## Impacto
Com RLS fraco, a chave do repositório dá acesso direto ao PostgREST do projeto. A senha 123456, se o seed
rodou em produção, é backdoor de Super Admin.

## Sugestão de correção
- Remover fallbacks hardcoded. Falhar o boot se `VITE_SUPABASE_URL` / `VITE_SUPABASE_ANON_KEY` faltarem ou
forem placeholder.
- Rotacionar a anon key no painel Supabase (o JWT atual está no git para sempre).
- Trocar a senha de `admin@frotamaq.com` no projeto hospedado; seeds de demo só em ambiente local, senha
forte e não commitada.
- Tirar `TEST_USER_PASSWORD` do código que o Vite empacota (`src/lib`).

## Critérios de aceite
- [ ] Build/dev sem VITE_SUPABASE_* aborta e não usa URL/key embutida
- [ ] grep no src/ não encontra eyJ... anon nem a URL aechddecxuvysrgpbord
- [ ] Login admin@frotamaq.com / 123456 falha no projeto hospedado
- [ ] Bundle de produção não contém 123456 como senha de teste

--- FIM ISSUE 5 ---

```

Issue 6 — [Segurança] Operações privilegiadas só escondidas na UI

Labels: **security, alta** · Achados F-12, F-15, F-17

Copie o bloco abaixo e cole numa issue do GitHub.

```

--- ISSUE 6 ---
## Título
[Segurança] Operações privilegiadas só escondidas na UI

## Labels sugeridas
`security, alta`

## Achados
F-12, F-15, F-17

## Descrição do problema e por que é explorável
O produto trata PermissionService / ProtectedRoute / flags `canManage` como autorização. Não existe API própria: o “servidor” é o RLS. Onde o caso de uso não checa papel e o RLS canônico também não checa, o botão oculto não impede a operação.

Caminhos verificados sem checagem de papel no application:
- `CreateVeiculoUseCase.execute` / `createVeiculo`
- `CreateManutencaoUseCase.execute` / `createManutencao`
- `deleteAllVeiculos` / `seedTestVeiculos` (só escopo de tenant)
- `veiculos_update_policy` no RLS RBAC permite ao mecânico atualizar qualquer coluna

## Evidência
`src/application/use-cases/VeiculoUseCases.ts:38-40`
```ts
async execute(input: NovoVeiculo) {
 const { data, error } = await this.veiculoRepo.create(input)
 return { data: data?.toDTO() ?? null, error }
}
```

`src/pages/VehiclesPage.tsx:124` – botão só se `canManageVehicles`.

`rls_rbac_policies.sql:143-150` – mecanico em UPDATE de veículos sem lista de colunas.

## Impacto
Mesmo que o RLS de papel seja corrigido depois, os use-cases continuam aceitando qualquer profile. Defesa em profundidade ausente. Mecânico altera placa/status além do km.

## Sugestão de correção
- Espelhar `usuario.podeGerenciarVeiculos()` / `podeRegistrarManutencao()` / `podeSeedTestData()` nos facades e use-cases antes do repositório.
- No RLS, restringir UPDATE do mecânico às colunas de medição (ou trigger que rejeita mudança de placa/empresa_id/status por mecanico).
- Manter ProtectedRoute só como UX.

## Critérios de aceite
- [ ] createVeiculo/createManutencao com perfil motorista retorna erro de permissão no application (mesmo se RLS estiver aberto)
- [ ] deleteAllVeiculos / seed com motorista retorna erro de permissão
- [ ] UPDATE de placa por mecanico é negado pelo RLS

--- FIM ISSUE 6 ---

```

Issue 7 — [Segurança] URL de foto de veículo sem allowlist de protocolo

Labels: **security**, **baixa** · Achados F-18

Copie o bloco abaixo e cole numa issue do GitHub.

```

--- ISSUE 7 ---
## Título
[Segurança] URL de foto de veículo sem allowlist de protocolo

## Labels sugeridas
`security, baixa`

## Achados
F-18

## Descrição do problema e por que é explorável
`foto_url` persistido no banco é interpolado em `

```